

I. Strategic Overview: The Imperative for Agentic Zero-Touch IAM

1.1 The Challenge: Automating Trust and Approvals

Modern enterprise security and operations are frequently burdened by manual processes in Identity and Access Management (IAM), resulting in significant latency during employee onboarding and non-compliance due to procedural errors or approval fatigue.¹ These delays hinder organizational velocity and increase the attack surface if access remains unnecessarily persistent or overly permissive. The proposed Zero-Touch IAM Agent directly addresses these challenges by automating the entire provisioning lifecycle, from approver identification and context generation to secure policy enforcement.

The fundamental objective of this solution is to eliminate manual intervention in routine access requests without compromising the principle of least privilege. Traditional automation often struggles with the cognitive aspects of security—specifically, generating policy justifications and accurately interpreting nuanced human communications. An agentic solution, leveraging the Google Agent Development Kit (ADK), provides the necessary framework for designing specialized workers that can handle context, state, tool integration, and advanced reasoning.²

1.2 Defining the Zero-Touch IAM Agent Value Proposition

To excel in a high-stakes environment like a security hackathon, the agent system must demonstrate innovation across three critical pillars, pushing standard automation into true autonomous decision support.

The first pillar is **High Autonomy**, achieved by using ADK's workflow agents, such as the SequentialAgent or LlmAgent, for dynamic routing and orchestration of complex, multi-step workflows.² This allows the system to manage prolonged, asynchronous tasks like waiting for human email responses. The second pillar is **Verifiable Security**, which moves beyond static policy enforcement to employ Just-in-Time (JIT) access principles, ensuring that the service accounts responsible for policy modifications possess the required privileges only for the

brief duration of the execution.⁴ This leverages specific GCP capabilities, including Custom IAM Roles to enforce the principle of Least Privilege (LPP).⁵ The third pillar is **Explainability (XAI) and Auditability**. Given the sensitivity of IAM changes, every decision made by the agent must be traceable and justified. This is achieved by using Retrieval Augmented Generation (RAG) to ground access justifications in official policy documents and logging all actions immutably via ADK logs integrated with Cloud Audit Logs.⁶

1.3 Key Use Cases and Prioritization

While the core functionality centers on automating the standard IAM approval process, an enterprise-grade solution must handle advanced scenarios. The primary and most prioritized use case is **New User Onboarding/Role Assignment**, which automatically grants baseline IAM roles.

Beyond baseline provisioning, two critical advanced use cases define the system's security maturity. The first is **Role Elevation Request (Advanced)**, where the agent processes requests for temporary, elevated access (e.g., granting the Owner role for a four-hour window). This scenario necessitates integration with JIT credentialing mechanisms and requires strict ADK tool access controls to prevent misuse.⁴ The second is **Policy Compliance Verification**. Here, the agent acts as a compliance officer, reviewing the proposed access request against current security documentation *before* granting access. This proactive verification requires the Policy Context Agent to utilize RAG (specifically, Vertex AI Search) to contextualize and justify the action, ensuring the grant is compliant with organizational policy mandates. This verification is extended to include dynamic, real-time constraints such as temporal access windows²² or geo-location restrictions (Improvement 2).⁹

Table 1: Agentic IAM Use Case Prioritization

Use Case	Description	Agentic Capability Utilized
Default Access Provisioning	Automatically grant baseline IAM roles upon joining a project/team after approver validation.	Approver Lookup, Communication, NLU Classification.
Role Elevation Request	A user requests temporary,	JIT Credentialing ⁴ , Custom

(Advanced)	elevated access that expires automatically.	Tool Access Controls. ⁸
Policy Compliance Verification	Agent reviews access against current security documentation <i>before</i> policy application.	RAG (Vertex AI Search) for Context Grounding, Context-Aware Temporal Checks . ²²

II. The Reference Architecture: GCP and ADK Integration Blueprint

2.1 Overall System Context Diagram and Data Flow

The Zero-Touch IAM Agent architecture is fundamentally an event-driven, serverless pipeline designed for security and scalability. The system is layered into four distinct components: the Event/Trigger Layer, the Orchestration/Reasoning Layer (ADK), the Data/Context Layer, and the Execution/Tooling Layer.

The data flow begins when a **Trigger** event (e.g., a new hire event from an external HR system or a direct API call) is sent to a Cloud Pub/Sub topic. This event triggers the **Orchestration** layer, specifically the Parent Agent (the IAM Orchestrator), which runs within a managed Cloud Run instance. This orchestrator handles session management, maintains state², and delegates specialized sub-tasks. It first accesses the **Data Lookups** layer, querying Cloud Firestore to identify the designated approvers and the default policy definitions.¹¹ This lookup informs the **Communication Loop**, where the Communication Agent sends the approval request. Crucially, the inbound response email is monitored via a separate service, which routes the classification result back to the Orchestrator via a dedicated Pub/Sub sink. Finally, upon verified approval, the Orchestrator initiates the **Execution** phase, where the highly restricted IAM Provisioning Agent uses a secure, custom tool to interact with the GCP IAM API, potentially utilizing temporary JIT credentials.

2.2 GCP Serverless Foundation: Deploying ADK Agents on Cloud Run

Cloud Run is selected as the definitive hosting environment for the core ADK agent framework due to its flexibility, scalability, and ability to handle containerized workloads.¹²

Unlike Cloud Functions, which offer simplicity but limit configuration options, Cloud Run supports custom containers.¹³ This container flexibility is essential for complex ADK deployments that require specific Python SDKs for the Gemini API, the ADK core library, and custom tools built on the Model Context Protocol (MCP).¹² Cloud Run ensures the ability to define the language runtime and web server environment, providing the necessary control for running sophisticated, custom-built agent services.¹³ Furthermore, deploying the agent container on Cloud Run provides an optimal balance of serverless scalability—scaling down to zero when idle to minimize costs¹²—and fine-grained environment control, which is necessary for a highly controlled security workflow.

The event-driven nature of the IAM process is perfectly suited for Cloud Run, which can be triggered directly by Cloud Pub/Sub events.¹⁴ Whether the event is the initial new user notification or the asynchronous notification of an approver's email response, Cloud Run rapidly scales up to handle the request, processes the specific ADK session, and then scales back down, ensuring efficiency and responsiveness. For persistent state and tool integration, Cloud Run serves as the underlying infrastructure layer, which can be further managed by platforms like the Vertex AI Agent Engine, offering managed scalability and integrated, secure tool execution.¹⁶

2.3 Data Strategy: Cloud Firestore for Dynamic Approver Mapping

Selecting the appropriate data store for organizational mappings, such as role-to-approver definitions and security prerequisites, is critical. This store must support rapid queries and accommodate frequent structural changes inherent in enterprise organizations.

The recommended solution is **Cloud Firestore**, a document database, over a relational solution like Cloud SQL.¹¹ IAM policies and organizational structures are inherently dynamic, often involving complex, hierarchical relationships (e.g., a specific role might require approval from three distinct groups: the Project Owner, the Department Manager, and a Security Officer). Firestore's document-based model, similar to JSON structures, easily handles this nested and variable data, allowing for flexible schema design that avoids the rigidity and overhead associated with managing complex schema updates in relational systems.³⁰

Firestore is designed as an enterprise-grade database, providing the necessary scalability, high availability, and low-latency synchronization required for a core system component that

is accessed for every IAM request.¹¹ This dynamic capability ensures the hackathon solution is not only performant but also capable of fast iteration and adaptation to changing organizational compliance rules.

III. Multi-Agent System Design using Google ADK

The IAM automation process is complex, involving multiple cognitive steps: lookup, contextualization, communication, classification, and execution. Google’s Agent Development Kit (ADK) provides the framework to structure this complexity using a specialized, hierarchical multi-agent system.¹⁷

3.1 ADK Orchestration Deep Dive: Defining Hierarchy and Specialization

The architecture employs a parent-child hierarchy defined by the BaseAgent structure, utilizing specialized agents for distinct roles.¹⁸ The core control mechanism is the **IAM Orchestrator**, which acts as the Parent Agent. This agent manages the overall session state, handles error recovery, and performs dynamic delegation using ADK's workflow mechanisms.²

The overall structure involves specialized sub-agents delegated specific tasks:

Table 2: ADK Multi-Agent Architecture for IAM

Agent Name	Primary Function (Reasoning Core)	Key Tools/Services Required	ADK Role Type
IAM Orchestrator	Manages session state, handles dynamic routing (LLM-driven transfer), and error recovery.	Configuration Store, Cloud Logging	LlmAgent / SequentialAgent [3]

Approver Lookup Agent	Retrieves required approvers and default policy based on user/role ID.	Cloud Firestore Tool , Caching Tool	FunctionTool Agent
Policy Context Agent	Uses RAG to find relevant policy documents and generate compliance justifications.	Vertex AI Search Tool (RAG Backend) ⁹	FunctionTool Agent
Communication Agent	Sends approval emails and monitors/triggers the inbound NLU processing.	Application Integration Tool (Email Connector) [31]	FunctionTool Agent
NLU Classifier Agent	Processes inbound email responses, classifies intent, and extracts structured data.	Vertex AI Gemini Tool (Structured Output) ¹⁹	LlmAgent
IAM Provisioning Agent	Securely executes the final policy application or removal via GCP IAM API calls.	Custom IAM Tool (API Wrapper), PAM Credential Manager ⁴	Execution Agent

3.2 The Core Workflow Agent Lifecycle (The 9-Step Request Flow)

The provisioning process follows a rigid, auditable nine-step lifecycle, orchestrated by the IAM Orchestrator (likely implemented as a SequentialAgent or a dynamically routing LlmAgent):

1. **Start (Event Trigger):** A Cloud Pub/Sub message signifying a new access request is received, initializing an ADK session.
2. **Delegate Lookup:** The Orchestrator calls the Approver Lookup Agent to retrieve the list

of required human approvers and the associated baseline IAM policy definition from Cloud Firestore.

3. **Delegate Context (RAG & Compliance Check):** The Orchestrator calls the Policy Context Agent. This agent queries the RAG backend to generate a justification, ensuring the request is pre-approved against policy documents (e.g., "Granting Role X is valid for new hires in the Engineering team per section 3.1 of the internal security manual"). Crucially, it performs **context-aware temporal checks** to ensure the request meets any time-of-day or location-based policy constraints (Improvement 2).
4. **Delegate Communication (Send):** The Orchestrator calls the Communication Agent, which sends the templated approval request email. This email includes the RAG-generated policy justification to provide necessary context to the human approver.
5. **Wait for Response:** The session state, handled robustly by ADK's Session Management feature ², persists the context and enters an asynchronous waiting state, pending the inbound approval signal.
6. **Inbound Trigger:** The approver's email response is captured by a separate service (e.g., a monitored mailbox), which publishes a response event containing the original Session ID to a Pub/Sub topic. This event triggers the next phase of the Orchestrator.
7. **Classification:** The Orchestrator delegates the email content to the NLU Classifier Agent, which performs structured analysis.
8. **Execute (Approval Path):** If the classification is "APPROVED" and the Policy Context Agent's compliance check passed, the Orchestrator initiates the highly secure execution phase: it requests temporary privilege elevation via Privileged Access Manager (PAM) ⁴, then calls the **IAM Provisioning Agent** to execute the policy binding update.
9. **Audit/Complete:** The Provisioning Agent logs its action and the RAG justification to the appropriate logging services.⁷ The ADK Session state is then officially closed.

This reliance on ADK's strong state and session management is fundamentally important for a production-grade IAM workflow. Since approval requests may take hours or days, the ADK session mechanism must reliably handle persistence, timeouts, and the asynchronous re-entry triggered by the inbound event, ensuring the correct context is instantaneously re-loaded before proceeding to the policy modification step. The stateless nature of the underlying Cloud Run infrastructure is masked by this stateful application architecture.

3.3 Implementation Guidance: Local Sub-Agents vs. External A2A Communication

The choice between running agents locally versus remotely dictates both performance and security boundaries.

For non-critical tasks like approver lookup, RAG context generation, and NLU classification, utilizing **Local Sub-Agents** that run within the same Cloud Run application process as the main Orchestrator is highly effective.³² This strategy maximizes performance, as communication occurs directly in memory, eliminating network overhead and latency associated with inter-service calls.³²

However, the **IAM Provisioning Agent** is unique in that it possesses the highest level of trust, handling sensitive GCP IAM API calls. For security purposes, it is best practice to encapsulate this component as a distinct, external A2A Service (Improvement 1).³³ This isolation enforces a critical layer of Defense in Depth, which is paramount for security workflows. By running the most privileged agent as a remote **A2A Server endpoint**, any potential security flaw or prompt injection attack within the LLM-driven Orchestrator is prevented from directly accessing the highly restricted service account keys or internal permissions required for IAM policy modification. The Orchestrator communicates with the IAM Provisioning Agent's A2A Server endpoint securely over HTTPS using the Agent2Agent (A2A) protocol (Improvement 1).³⁴ A2A standardizes this communication, ensuring interoperability and secure message passing between independent services, regardless of their internal ADK framework implementation.²⁰ This separation ensures that the core reasoning agent cannot directly execute privileged actions without going through a strictly defined and firewalled communication channel.

IV. Secure IAM Tooling and Execution Layer

The modification of IAM policies is the most sensitive operation performed by the agent. The execution layer must be built around rigorous adherence to the Least Privilege Principle (LPP) and Zero Trust architecture.⁸

4.1 Building the Custom IAM Tool: Encapsulating GCP API Calls

The **IAM Provisioning Agent** requires interaction with the GCP IAM API to apply the policy changes. Since direct, unmediated LLM access to sensitive APIs is a significant security risk, a custom tool must be created.

This Custom IAM Tool will function as a secure, authorized wrapper around the necessary GCP API methods, primarily `cloudresourcemanager.projects.setIamPolicy`.²³ It is implemented as a Function Tool wrapper within ADK, or preferably as a dedicated Model Context Protocol

(MCP) Server deployed on its own secure Cloud Run instance (Improvement 1).¹² This encapsulation ensures that all IAM actions are strictly defined, validated, and parameterized within the wrapper. The LLM's role is thus restricted to calling the predefined function with validated inputs, preventing it from generating and executing arbitrary or malicious API calls.

4.2 Least Privilege Implementation: Custom IAM Roles

Rigid enforcement of LPP dictates that the service account executing the policy change must possess the minimum permissions necessary for its function.²⁵ The use of default service accounts or blanket project-level access is strictly prohibited as per security best practices.²⁴

The service account responsible for provisioning (\$SA_IAM_Provisioner\$) will be assigned a highly specific **Custom IAM Role**.⁵ This custom role will contain only the essential permissions required for reading and applying policies: iam.policy.get (to read the current state before modification) and iam.policy.set (to apply the new approved binding). This minimizes the potential blast radius should the service account credentials ever be compromised.

4.3 Advanced Security: Integrating with Privileged Access Manager (PAM) for Just-in-Time (JIT) IAM

The transition from a service account with permanent, restricted access to one with truly ephemeral, controlled access represents a major architectural leap, aligning with Zero Trust standards.²² This is achieved by integrating the execution flow with GCP's Privileged Access Manager (PAM), currently available in Preview.⁴

Under this JIT workflow:

1. The \$SA_IAM_Provisioner\$ holds virtually zero policy modification privileges in its baseline state.
2. Upon verified human approval and successful policy compliance checks, the IAM Provisioning Agent initiates a request to PAM.
3. PAM temporarily grants the required Custom IAM Role (containing iam.policy.set) to the service account for a short, controlled window, such as five minutes.⁴
4. The agent uses the short-lived, elevated token to successfully execute the iam.policy.set API call.
5. The temporary privilege automatically expires, returning the service account to its

non-privileged baseline state.

This approach ensures that even if an attacker compromises the service account's long-term identity, they gain no ability to modify production IAM policies because the required privilege is automatically revoked after the brief operational window.

4.4 Service Account Isolation: Enforcing Separation of Duties

The security posture is further strengthened by enforcing separation of duties across the agent system components.²⁵

The system uses three primary, isolated service accounts:

1. **\$SA_IAM_Provisioner\$**: Exclusively used by the A2A Provisioning Service (Improvement 1). Its sole permission, `iam.policy.set`, is only active via JIT elevation.⁴ This account is protected by strict policies against user-managed keys.²⁴
2. **\$SA_NLU\$**: Dedicated to the NLU Classifier Agent. It only possesses the permissions necessary for invoking the Vertex AI/Gemini API (`vertexai.modelUser`). It has no capability to modify infrastructure or IAM policies.
3. **\$SA_RAG\$**: Used by the Policy Context Agent. Its permissions are restricted to retrieving data from the Vertex AI Search index (`vertexai.datastoreUser`).¹⁰

The architectural implication of this separation is profound: a successful attack on a lower-privileged agent (e.g., a prompt injection against the NLU agent) grants the attacker zero ability to execute the critical security modification API calls, as the NLU agent's service account (`$SA_NLU$`) lacks the necessary permissions, and the privileged service account (`$SA_IAM_Provisioner$`) is protected by JIT access control boundaries.²⁵

V. Generative AI for Intelligent Decision Support

Generative AI is integrated not merely for conversational ability but to provide deterministic, high-reliability decision support, particularly in three key areas: efficient intent classification, contextual policy validation, and grounding decisions in compliance policy.

5.1 NLU for Approver Response Processing: Structured Output and

Efficiency

The primary point of failure in automated approval processes is the ambiguity of human communication. To eliminate the risk of misinterpreting an approver's email response, the **NLU Classifier Agent** must enforce structured output using the Gemini API in Vertex AI.²⁶

The NLU Classifier Agent sends the raw email text to the **Gemini 2.5 Flash** model. This is an explicit choice (Improvement 3) because Gemini Flash offers high speed and efficiency for low-latency classification tasks, reducing processing cost and time per request.³⁵ Crucially, the prompt enforces a strict JSON schema defined by the developer, utilizing the model's structured output capability.¹⁹ This guarantees that the downstream Orchestrator receives a machine-readable, reliable signal rather than raw, unstructured text.

For example, the model must return a mandatory JSON object structure:

JSON

```
{  
  "status": "APPROVAL_STATUS",  
  "approver_id": "string",  
  "reason_summary": "string"  
}
```

The APPROVAL_STATUS field is constrained by an enum to one of three states: APPROVED, DENIED, or REJECTED_CONTEXT_MISSING. The prompt engineering must specifically instruct Gemini to select REJECTED_CONTEXT_MISSING if the response is ambiguous, non-committal, or if the intent cannot be determined with high confidence. This constraint is vital for preventing the agent from making risky automated approvals based on uncertainty.

5.2 RAG for Policy Justification and Context-Aware Constraints

Compliance and auditability require that every access decision be justified by organizational policy. The **Policy Context Agent** utilizes Retrieval Augmented Generation (RAG) and extends it with dynamic validation logic to fulfill this Explainability (XAI) requirement (Improvement 2).²⁷

The RAG backend is implemented using Vertex AI Search (RAG Engine), which provides enhanced performance and scalability for vector search across private data sources.⁹ Organizational policy manuals, security handbooks, and role definitions are ingested from Cloud Storage, chunked, and vectorized using the Vertex AI Embeddings API (e.g., gemini-embedding-001).³⁷

The Policy Context Agent executes the retrieval flow, moving beyond static justification:

1. **Context Retrieval:** It queries the Vertex AI Search index to retrieve relevant policy document chunks pertaining to the access request.²⁸
2. **Compliance Validation (Improvement 2):** Before passing the retrieved context to the LLM for justification, the agent checks for dynamic, context-aware constraints within the retrieved policy. For example, if the policy dictates the elevated role can only be granted during business hours (9 AM - 5 PM) or from a specified geographical location (if location data is available), the Policy Context Agent explicitly checks the current system time or request source against these constraints. This check ensures the policy application is **fine-grained and context-aware**.²²
3. **Grounded Justification:** If the check passes, the agent passes the retrieved policies and the request context to the LLM (likely a larger, more capable model than Flash, such as Gemini 2.5 Pro, to handle complex synthesis) to generate a concise, grounded justification.
4. **Failure Signal:** If the compliance validation fails (e.g., the request is outside the approved time window), the agent generates a "REJECTED_POLICY_VIOLATION" signal, preventing the Orchestrator from proceeding to the Provisioning Agent.

This robust, grounded justification serves two functions: it provides necessary context to the human approver and it is logged immutably, forming the core of the explainability record for compliance audits.²⁸

VI. Auditing, Explainability (XAI), and Compliance Framework

For any system modifying critical infrastructure components like IAM, the design must prioritize transparency, auditability, and clear explainability of agent actions, aligning with frameworks like NIST AI RMF.⁶

6.1 Immutable Logging Architecture: Agent Events and Cloud Audit

Logs

A dual-layer logging strategy is mandatory, ensuring both the agent's internal reasoning and the final execution steps are captured and correlated.⁷

The first layer consists of **ADK Internal Logs**. The ADK framework must be configured to capture every relevant event in the workflow: initial inputs, session state transitions, LLM tool inputs/outputs, and agent delegation calls.⁶ This record captures the detailed *agent reasoning chain*.

The second layer is **GCP Cloud Audit Logs**. Any modification to an IAM policy (the `iam.policy.set` operation) is automatically logged by GCP as an Admin Activity Audit Log.⁷ The key architectural requirement is to integrate the ADK internal log data (including the final RAG justification narrative and the **context-aware compliance validation result**) into a secure Log Sink destination, such as a BigQuery dataset or Cloud Storage bucket.²⁹ A unique Session ID must be used as the correlation key, allowing compliance teams to cross-reference the ADK log events (the 'why' and 'how' of the decision) with the Cloud Audit Log entry (the 'what' of the execution).⁶ Access to these sensitive forensic logs must be strictly controlled using IAM permissions on log views and sink destinations.⁷

6.2 Transparency in Automation: Generating Human-Readable Audit Narratives

Explainability (XAI) requires translating the complex, multi-agent logic into a straightforward, human-readable narrative.⁶ The IAM Orchestrator is responsible for synthesizing this summary upon successful session completion.

This final structured summary serves as the official record: *“Transaction ID: On, IAM Orchestrator received request for User X to get Role Y. Approver Z approved via email (NLU Confidence 98%). **Policy Context Agent verified temporal policy (Check Passed)**. Provisioning Agent successfully executed `iam.policy.set` using JIT elevation. Action grounded by policy:”*

This narrative is stored as a custom log entry, satisfying the requirement for clear audit-ready reporting and evidence collection.⁶

6.3 Security Controls for Agentic Systems (ADK Guardrails)

The security of an agentic system depends on embedding controls directly into the architecture.⁸

- **Verifiable Agent Identity:** Every agent must have a verifiable identity linked to its dedicated service account.⁸
- **Tool Access and Sandboxing:** The Custom IAM Tool used by the Provisioning Agent must operate within a tightly controlled environment, restricting its execution context to only the approved API calls to prevent the LLM from attempting to access unauthorized functions.⁸
- **Input Validation:** A critical guardrail is strict input validation and sanitization performed by the NLU Classifier Agent *before* the email content is passed to Gemini. This mitigates prompt injection attacks, where malicious actors attempt to manipulate the LLM's instructions to force a deceptive "APPROVED" structured output.⁸
- **Secure A2A Communication:** The highest-privileged interaction (Orchestrator to Provisioning Agent) is secured by the A2A protocol and IAM authentication, protecting the critical execution step (Improvement 1).³⁴

6.4 Designing the Fail-Safe: Emergency Stop and Human Oversight

Because the agent modifies high-privilege IAM resources, an immediate safety mechanism is non-negotiable.⁸

An **Emergency Stop** mechanism must be implemented, possibly as a monitored API endpoint exposed by the Orchestrator's Cloud Run service. Activation of this endpoint would immediately pause all active IAM provisioning sessions and, critically, initiate a bulk revocation of all active JIT tokens managed by PAM, ensuring the \$SA\IAM_Provisioner\$ instantly loses all ability to execute policy changes.

Furthermore, a **Human-in-the-Loop** control is required for ambiguous inputs. If the NLU Classifier Agent returns the non-deterministic REJECTED_CONTEXT_MISSING enum, or if the Policy Context Agent returns REJECTED_POLICY_VIOLATION, the Orchestrator must immediately halt the automated workflow and escalate the request to a human analyst for manual review, preventing any risky, uncertain automated decisions from proceeding to policy enforcement.

VII. Hackathon Submission Strategy and Future Roadmap

7.1 Immediate High-Impact Features for the Hackathon

To maximize impact in a hackathon setting, the presentation must focus on showcasing complexity, security maturity, and the deterministic use of AI.

1. **Isolated JIT IAM Execution via A2A (Improvement 1):** This feature is the highest-value differentiator. Demonstrating the separation of the LLM-driven Orchestrator from the privileged **IAM Provisioning Agent** (running as a distinct A2A Server) that then integrates with GCP PAM to request temporary, short-lived elevation, executes the policy change, and reverts to zero privilege showcases unparalleled architectural maturity.²⁰
2. **Context-Aware Compliance Guardrail (Improvement 2):** Showcasing the Policy Context Agent actively blocking an access request (e.g., denying an elevation request submitted at 11 PM because policy dictates "business hours access only") and logging the "REJECTED_POLICY_VIOLATION" signal proves the agent is an intelligent, context-aware policy enforcement mechanism, not just an executor.²²
3. **Deterministic NLU via Structured Output:** Visual proof of the NLU Classifier Agent reliably transforming ambiguous human email text into a predefined, strict JSON schema (using Gemini Structured Output) proves that the AI component is reliable enough for security workflows.¹⁹
4. **Audit Trail Visualization:** Presenting a simple dashboard that visually correlates the ADK's internal decision path (including the RAG justification and compliance check result) with the immutable setIamPolicy entry in Cloud Audit Logs demonstrates end-to-end transparency and compliance readiness.⁷

7.2 Recommendations for Feature Expansion (Roadmap)

The robust architectural foundation allows for strategic future expansion into broader security automation challenges:

- **Automated JIT Access Revocation:** The system should evolve from merely granting temporary access to actively monitoring the user's activity and automatically revoking temporary roles based on time limits or deviation from baseline behavioral patterns. This

enforces true Just-in-Time, Just-Enough Access (JIT JEA) and aligns with advanced identity governance models.²²

- **Alert Fatigue Reduction:** The agent's contextual awareness (built on RAG) can be applied to security monitoring. The system can automatically filter, prioritize, or even suppress alerts related to newly provisioned roles by cross-referencing the access context with known threat intelligence or historical incident data, freeing security analysts from routine triage.
- **Intelligent Model Routing (Improvement 3):** Implement model routing logic to ensure cost and latency efficiency. The fast, cost-effective Gemini 2.5 Flash model should be exclusively used for high-frequency, low-complexity tasks (like NLU classification), reserving the larger, more powerful foundation models (like Gemini 2.5 Pro) only for complex tasks such as RAG justification generation and advanced compliance checks. This showcases production readiness and efficiency awareness.

7.3 Measuring Success: Key Performance Indicators (KPIs)

Success must be quantified using metrics that demonstrate real-world security and operational improvements:

Table 3: Key Performance Indicators for the Agentic IAM System

KPI Category	Metric	Definition
Velocity & Efficiency	Mean Time to Provision (MTTP)	Measures time from initial request trigger to policy enforcement completion. (Target: Reduction by >80% compared to manual process).
Security & Compliance	JIT Token Usage Frequency	Measures the percentage of policy modifications executed using JIT elevated credentials versus permanent access. (Target: 100% JIT execution).
Accuracy	NLU Classification Error	Percentage of human email

	Rate	responses misclassified by the NLU Agent. (Target: $<0.5\%$ error rate due to Structured Output enforcement).
Auditability	Compliance Traceability Score	Measures the success rate of correlating ADK internal log events with corresponding Cloud Audit Logs via Session ID. (Target: 100% trace correlation).

VIII. Conclusions

The proposed Zero-Touch IAM Provisioning Agent establishes an expert-level, enterprise-grade architecture leveraging the strengths of the Google Cloud Platform and the Agent Development Kit (ADK). By adopting a specialized, multi-agent hierarchy and strictly enforcing the **Agent2Agent (A2A) isolation boundary** for the most privileged components, the solution decomposes a complex security workflow into manageable, auditable components with robust defense in depth.

The integration of advanced Generative AI capabilities—specifically Vertex AI Search for policy grounding (RAG), **context-aware compliance checks**, and Gemini Structured Output for deterministic NLU classification—moves the system beyond simple task automation into true cognitive decision support. The architectural commitment to isolation (A2A) and Just-in-Time privilege elevation (PAM) sets a high standard for verifiable security and least privilege enforcement.

This blueprint not only meets the immediate requirements for automating IAM approvals but provides a foundation for future security operations, ensuring that velocity, security, explainability, and compliance are simultaneously prioritized throughout the entire lifecycle of access management. The resulting system delivers a high-fidelity, scalable, and secure solution suitable for the rigorous demands of an enterprise security hackathon.

Works cited

1. 7 Use Cases for AI Agents in Cybersecurity - Jit.io, accessed October 30, 2025,

<https://www.jit.io/resources/ai-security/7-use-cases-for-ai-agents-in-cybersecurity>

2. Agent Development Kit (ADK) - Google, accessed October 30, 2025, <https://google.github.io/adk-docs/get-started/about/>
3. Agent Development Kit - Google, accessed October 30, 2025, <https://google.github.io/adk-docs/>
4. IAM release notes | Identity and Access Management (IAM) - Google Cloud Documentation, accessed October 30, 2025, <https://docs.cloud.google.com/iam/docs/release-notes>
5. Roles and permissions | Identity and Access Management (IAM) | Google Cloud Documentation, accessed October 30, 2025, <https://docs.cloud.google.com/iam/docs/roles-overview>
6. Rise of Agentic AI Security: Protect Workflows, Not Just Apps - Reco AI, accessed October 30, 2025, <https://www.reco.ai/blog/rise-of-agentic-ai-security>
7. Best practices for Cloud Audit Logs, accessed October 30, 2025, <https://docs.cloud.google.com/logging/docs/audit/best-practices>
8. Agentic AI security: Complete guide to threats, risks & best practices 2025 - Rippling, accessed October 30, 2025, <https://www.rippling.com/blog/agentic-ai-security>
9. Vertex AI RAG Engine: Build & deploy RAG implementations with your data - Google Cloud, accessed October 30, 2025, <https://cloud.google.com/blog/products/ai-machine-learning/introducing-vertex-ai-rag-engine>
10. RAG infrastructure for generative AI using Vertex AI and Vector Search | Cloud Architecture Center, accessed October 30, 2025, <https://docs.cloud.google.com/architecture/gen-ai-rag-vertex-ai-vector-search>
11. Choose a Database: Cloud Firestore or Realtime Database - Firebase, accessed October 30, 2025, <https://firebase.google.com/docs/database/rtdb-vs-firestore>
12. Build and deploy an ADK agent that uses an MCP server on Cloud Run | Google Codelabs, accessed October 30, 2025, <https://codelabs.developers.google.com/codelabs/cloud-run/use-mcp-server-on-cloud-run-with-an-adk-agent>
13. Cloud run vs cloud functions for serverless | Google Cloud Blog, accessed October 30, 2025, <https://cloud.google.com/blog/products/serverless/cloud-run-vs-cloud-functions-for-serverless>
14. Cloud Pub/Sub trigger | Application Integration - Google Cloud Documentation, accessed October 30, 2025, <https://docs.cloud.google.com/application-integration/docs/configure-pubsub-trigger>
15. Event-driven architecture with Pub/Sub | Solutions - Google Cloud, accessed October 30, 2025, <https://cloud.google.com/solutions/event-driven-architecture-pubsub>
16. Deploy Your ADK Agent to Vertex AI Agent Engine: A Complete Tutorial | by Aryan Irani | Google Cloud - Community | Oct, 2025 | Medium, accessed October 30,

- 2025,
<https://medium.com/google-cloud/deploy-your-adk-agent-to-vertex-ai-agent-engine-a-complete-tutorial-2975e2229580>
17. Build multi-agentic systems using Google ADK | Google Cloud Blog, accessed October 30, 2025,
<https://cloud.google.com/blog/products/ai-machine-learning/build-multi-agentic-systems-using-google-adk>
 18. Multi-Agent Systems in ADK - Google, accessed October 30, 2025,
<https://google.github.io/adk-docs/agents/multi-agents/>
 19. Structured output | Generative AI on Vertex AI | Google Cloud, accessed October 30, 2025,
<https://cloud.google.com/vertex-ai/generative-ai/docs/multimodal/control-generated-output>
 20. Architecting the Future: A Deep Dive into Google's ADK and A2A for Multi-Agent Systems, accessed October 30, 2025,
<https://medium.com/genusoftechnology/architecting-the-future-a-deep-dive-into-googles-adk-and-a2a-for-multi-agent-systems-5ebfbc16d714>
 21. Getting Started with Agent2Agent (A2A) Protocol: A Purchasing Concierge and Remote Seller Agent Interactions on Cloud Run and Agent Engine | Google Codelabs, accessed October 30, 2025,
<https://codelabs.developers.google.com/intro-a2a-purchasing-concierge>
 22. Agentic AI Identity & Access Management | CSA - Cloud Security Alliance, accessed October 30, 2025,
<https://cloudsecurityalliance.org/artifacts/agentic-ai-identity-and-access-management-a-new-approach>
 23. IAM roles and permissions | Cloud Deploy - Google Cloud Documentation, accessed October 30, 2025,
<https://docs.cloud.google.com/deploy/docs/iam-roles-permissions>
 24. 24 Google Cloud Platform (GCP) security best practices - Sysdig, accessed October 30, 2025,
<https://www.sysdig.com/learn-cloud-native/24-google-cloud-platform-gcp-security-best-practices>
 25. Best practices for using service accounts | Identity and Access Management (IAM) | Google Cloud Documentation, accessed October 30, 2025,
<https://docs.cloud.google.com/iam/docs/best-practices-service-accounts>
 26. Natural Language AI - Google Cloud, accessed October 30, 2025,
<https://cloud.google.com/natural-language>
 27. What is RAG? - Retrieval-Augmented Generation AI Explained - AWS - Updated 2025, accessed October 30, 2025,
<https://aws.amazon.com/what-is/retrieval-augmented-generation/>
 28. Retrieval Augmented Generation - Architecture Patterns - IBM, accessed October 30, 2025, <https://www.ibm.com/architectures/patterns/genai-rag>
 29. Best practices for working with Google Cloud Audit Logs, accessed October 30, 2025,
<https://cloud.google.com/blog/products/management-tools/best-practices-for->

[working-with-google-cloud-audit-logging](#)

30. Compare Google Cloud Firestore vs. Cloud SQL - G2, accessed October 30, 2025, <https://www.g2.com/compare/google-cloud-firestore-vs-google-cloud-sql>
31. Introduction to A2A - Agent Development Kit - Google, accessed October 30, 2025, <https://google.github.io/adk-docs/a2a/intro/>
32. Vertex AI Agent Builder | Google Cloud, accessed October 30, 2025, <https://cloud.google.com/products/agent-builder>
33. Specification - A2A Protocol, accessed October 30, 2025, <https://a2a-protocol.org/latest/specification/>
34. Analyze and label Gmail messages with Gemini and Vertex AI | Google Workspace add-ons, accessed October 30, 2025, <https://developers.google.com/workspace/add-ons/samples/gmail-sentiment-analysis-ai>
35. Structured output | Gemini API - Google AI for Developers, accessed October 30, 2025, <https://ai.google.dev/gemini-api/docs/structured-output>
36. Get text embeddings | Generative AI on Vertex AI - Google Cloud Documentation, accessed October 30, 2025, <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings>